



PROJECT REPORT

Secure Text Caesar Cipher Encryption & Decryption

Submitted To:

Sir Adeel Amjad

Submitted By:

Name	Roll Number	GitHub Links
Sadia Eman Abdul Ghafoor	SU92-BSDSM-F23-019	https://github.com/Sdia1029/SecureText-Caesar-Cipher-COAL-Project
Zunaira Abdul Shakoor	SU92-BSDSM-F23-034	zunaria15/SecureText-COAL-Project · GitHub

Section: BSDS-6A

Department: Software Engineering

Table of Contents

1. Introduction	3
2. Objectives	3
3. Tools & Environment	4
4. COAL Concepts Used	4
5. Program Features	6
6. Code Explanation	7
7. Assembly Commands Reference	12
8. Output / Results	14
9. Conclusion	15

1. Introduction

SecureText is a fully functional Caesar Cipher Encryption and Decryption tool developed in 16-bit x86 Assembly Language using the NASM assembler and executed in the DOSBox environment. This project was developed as part of the Computer Organization and Assembly Language (COAL) course at Superior University, Department of Software Engineering.

The Caesar Cipher is one of the oldest and simplest encryption techniques in the history of cryptography. It works by shifting each letter of the plaintext by a fixed number of positions forward in the alphabet. For example, with a shift of 2, the letter 'A' becomes 'C', 'B' becomes 'D', and so on. To decrypt, the shift is simply reversed.

This project implements the Caesar Cipher entirely in assembly language, demonstrating core COAL concepts such as register manipulation, memory addressing, interrupt handling, control flow using conditional jumps, stack operations, and modular string processing — all at the machine level without any high-level language constructs.

2. Objectives

The main objectives of this project are:

- Implement Caesar Cipher encryption and decryption in 16-bit x86 Assembly Language
- Demonstrate practical use of COAL concepts: registers, interrupts, memory, loops, and procedures
- Provide an interactive menu-driven interface for the user
- Implement Brute Force attack to try all 25 possible keys
- Show Step-by-Step encryption process to visualize how each character is shifted
- Analyze text statistics: uppercase, lowercase, spaces, and other characters
- Produce a working COM file that runs correctly in DOSBox (real 16-bit environment)

3. Tools & Environment

Tool / Software	Version	Purpose
NASM (Netwide Assembler)	0.98.38	Assembling .asm source code into .com binary
DOSBox	0.74+	Running 16-bit COM files in a DOS environment
Windows	10 pro	Host operating system
Text Editor (Notepad++)	Any	Writing and editing assembly source code

How to Assemble and Run

Step 1: Write the source code and save as CipherASM.asm

Step 2: Open DOSBox and navigate to the folder containing the file

Step 3: Assemble using the following command: **nasm**

CipherASM.asm -o CipherASM.com Step 4: Run the program:

CipherASM.com

4. COAL Concepts Used

This project covers a wide range of COAL (Computer Organization & Assembly Language) concepts. Each concept is explained below with its usage in the project.

4.1 Registers

Registers are the fastest storage locations inside the CPU. This project uses all major 16-bit x86 registers:

Register	Type	Usage in Project
AX / AL / AH	General Purpose (Accumulator)	Storing characters, arithmetic, function return values
BX / BL / BH	General Purpose (Base)	Temporary storage (e.g., storing shifted character in BL)
CX / CL	General Purpose (Counter)	Loop counter for iterating over string characters
DX / DL	General Purpose (Data)	Printing characters via INT 21h, passing string address
SI	Source Index	Pointer to traverse input_buf character by character
DI	Destination Index	Pointer to write into work_buf during copy operation
DS / ES / CS	Segment Registers	Set at startup to point to code segment (COM file)
SP	Stack Pointer	Managed automatically when using PUSH/POP instructions

4.2 Memory & Data Segment

In a COM file, code and data share the same 64KB segment. All variables are declared in the data section before the program code:

```
input_buf db 51, 0 ; buffer: max 51 chars, actual count stored here
times 52 db 0 ; 52 bytes of actual character storage work_buf times
54 db 0 ; working buffer for brute force copies shift_key db 0 ;
stores the shift value (1-25) str_len db 0 ; stores actual length
of entered string
```

4.3 Interrupts (INT)

Interrupts are signals that pause the CPU to run a special service routine. This project uses DOS and BIOS interrupts extensively:

Interrupt	AH Value	Function	Used For
INT 21h	09h	Print String (ends with \$)	Printing all messages and output text
INT 21h	01h	Read Character with Echo	Reading user key press from keyboard
INT 21h	0Ah	Buffered Keyboard Input	Reading full text string from user
INT 21h	4Ch	Terminate Program	Exiting the program cleanly
INT 10h	02h	Write Char via DL	Character output in step-by-step mode

4.4 Control Flow (Jumps & Conditionals)

Assembly has no if/else or for loops. All control flow is done using CMP (compare) and jump instructions:

Instruction	Meaning	Example Use in Project
CMP al, 'A'	Compare AL with ASCII value of A	Check if character is uppercase letter
JL label	Jump if Less (signed)	Jump if char < 'A' (not uppercase)
JG label	Jump if Greater (signed)	Jump if char > 'Z' (not uppercase)

JE label	Jump if Equal (ZF=1)	Check if user pressed '6' for exit
JNE label	Jump if Not Equal	Menu routing: skip option if not matched
JGE label	Jump if Greater or Equal	Brute force loop exit when ctr >= 26
JMP label	Unconditional Jump	Loop back, retry input, etc.
LOOP label	Decrement CX, jump if CX != 0	Main character-by-character processing loop

4.5 Stack Operations (PUSH / POP)

The stack is used to temporarily save register values before a subroutine call and restore them after. This is critical in the Step-by-Step feature to prevent print functions from overwriting AL:

```
push ax      ; Save original character before calling print functions
              ; (print functions overwrite AX/AL internally) call
print_str ; This modifies AH and possibly AL pop ax      ;
Restore original character after call call shift_char; Now we
can safely use the original character
```

4.6 Procedures (Subroutines)

Assembly procedures are reusable blocks of code called with CALL and returned from with RET. This project uses a modular design with over 15 procedures:

Procedure Name	Purpose
print_banner	Prints the welcome banner at startup
print_menu	Displays the main menu options
get_text	Reads buffered string input from user using INT 21h/0Ah
get_key	Reads shift key (1-25) with validation
cipher_inbuf	Encrypts/decrypts text in input_buf using shift_key
cipher_workbuf	Encrypts a copy in work_buf (used by brute force)

shift_char	Core cipher logic: shifts a single character by shift_key
copy_buf	Copies input_buf to work_buf using REP MOVSB
do_encrypt	Handles option 1: full encrypt flow
do_decrypt	Handles option 2: full decrypt flow (reverses key)
do_brute	Handles option 3: tries all 25 keys
do_stepbystep	Handles option 4: shows each character being shifted
do_stats	Handles option 5: counts character categories
print_num	Prints a number (0-99) to screen using division
print_str / print_nl	Utility: print string or newline

4.7 String Operations

The project uses REP MOVSB for fast memory copy from input_buf to work_buf:

copy_buf:

```

    mov si, input_buf+2 ; source: actual text starts at offset +2
    mov di, work_buf    ; destination: work buffer    xor cx, cx
; clear CX
    mov cl, [str_len]   ; CX = number of characters to copy  inc cx
; +1 to also copy the '$' terminator  rep movsb           ; repeat
MOVSB: [SI]->[DI], SI++, DI++, CX--  ret

```

4.8 ASCII & Modular Arithmetic

The core cipher uses ASCII arithmetic and modular math to wrap around the alphabet:

; Formula: encrypted = ((char - 'A') + shift) % 26 + 'A'

```

shift_char:           ; Input: AL = character to shift
sub al, 'A'           ; AL = 0-25 (A=0, B=1 ... Z=25)
add al, [shift_key]   ; AL = position + shift  cmp
al, 26                ; did we go past Z?

```

```

    jl  sc_up_ok      ; if AL < 26 we are fine
    sub al, 26        ; else wrap around (% 26)
sc_up_ok:
    add al, 'A'       ; convert back to ASCII
ret

```

5. Program Features

Option	Feature	Description
[1]	Encrypt Text	Takes plaintext and shift key, outputs Caesar-encrypted ciphertext
[2]	Decrypt Text	Takes ciphertext and shift key, reverses cipher by applying (26 - key)
[3]	Brute Force	Tries all 25 possible shift keys and displays each decryption result
[4]	Step-by-Step Encrypt	Shows each letter being shifted individually with its result
[5]	Text Statistics	Counts total chars, uppercase, lowercase, spaces, and other symbols
[6]	Exit	Exits the program with a goodbye message

5.1 Encryption Logic

To encrypt, the program shifts each alphabetic character forward in the alphabet by the `shift_key` amount. Nonalphabetic characters (spaces, numbers, symbols) are passed through unchanged.

Example: PAKISTAN with shift = 2

Original	ASCII	+ Shift	% 26	Encrypted
P	80	$17 + 2 = 19$	19	R
A	65	$0 + 2 = 2$	2	C
K	75	$10 + 2 = 12$	12	M
I	73	$8 + 2 = 10$	10	K
S	83	$18 + 2 = 20$	20	U

T	84	$19 + 2 = 21$	21	V
A	65	$0 + 2 = 2$	2	C
N	78	$13 + 2 = 15$	15	P

5.2 Decryption Logic

Decryption is done by reversing the shift. The program adds $(26 - \text{key})$ instead of the key itself. This is equivalent to shifting backwards:

```
; Decryption key = 26 - shift_key
; Example: to decrypt with key=2, encrypt with key=24 mov
al, 26
sub al, [shift_key] ;  $AL = 26 - 2 = 24$  mov
[shift_key], al ; store reversed key
call cipher_inbuf ; run encryption with reversed key
```

5.3 Brute Force

The brute force feature iterates through all 25 possible shift values and prints the decrypted text for each. This is a classic cryptanalysis technique for breaking Caesar Cipher without knowing the key:

```
bf_loop:
    mov al, [brute_ctr] ; current key being tried (1 to 25)
    cmp al, 26
    jge bf_end ; stop after key 25
    call copy_buf ; fresh copy of ciphertext for each attempt
    mov al, 26
    sub al, [brute_ctr] ; reverse the key
    mov [shift_key], al
    call cipher_workbuf ; decrypt the copy
    call print_wbuf ; print result inc byte
    [brute_ctr] ; next key jmp bf_loop
```

6. Code Explanation

6.1 Program Entry Point

This is a 16-bit COM file. The ORG 0x100 directive tells NASM that the program starts at offset 0x100 (DOS loads COM files at offset 100h, reserving the first 256 bytes for the PSP — Program Segment Prefix).

```
[BITS 16]      ; Tell NASM to generate 16-bit code
[ORG 0x100]    ; COM files load at offset 100h
```

```
jmp start    ; Jump over data section to code
```

start:

```
mov ax, cs    ; Get code segment value  mov ds, ax    ; Set
DS = CS (data and code in same segment)  mov es, ax    ; Set
ES = CS (needed for MOVSB string copy)  call print_banner
```

6.2 Input Handling (Buffered Input)

The program uses INT 21h function 0Ah for buffered keyboard input. This allows reading a full string including spaces: `get_text`:

```
mov dx, enter_text    ; print the prompt  call print_str
mov ah, 0Ah           ; DOS function: buffered input  mov
dx, input_buf         ; DX points to input buffer structure  int
21h                  ; DOS reads text until Enter is pressed
                    ; input_buf+0 = max allowed chars (51)
                    ; input_buf+1 = actual chars entered (auto-filled by DOS)
input_buf+2 onwards = the actual text characters  mov al,
[input_buf+1]; load actual length  mov [str_len], al    ; save
it  xor ah, ah  mov bx, input_buf+2  add bx, ax        ;
BX = address after last character  mov byte [bx], '$'   ; place
string terminator for INT 21h/09h  ret
```

6.3 Core Shift Function

`shift_char` is the heart of the cipher. It handles both uppercase and lowercase letters, leaving all other characters unchanged:

```
shift_char:          ; Input: AL = character, [shift_key] = shift amount
cmp al, 'A'
jl  sc_low           ; below 'A': check lowercase
cmp al, 'Z'
jg  sc_low           ; above 'Z': check lowercase
; --- Uppercase ---
sub al, 'A'          ; normalize: A=0, B=1 ... Z=25
add al, [shift_key]  cmp al, 26  jl  sc_up_ok
```

```

    sub al, 26      ; wrap around past Z sc_up_ok:
    add al, 'A'     ; back to ASCII
ret sc_low:
    cmp al, 'a'
    jl sc_end      ; not a letter: return unchanged
cmp al, 'z'  jg sc_end ; --- Lowercase --- sub
al, 'a'  add al, [shift_key]  cmp al, 26  jl
sc_lo_ok  sub al, 26 sc_lo_ok:
    add al, 'a' sc_end:
    ret

```

7. Assembly Commands Reference

7.1 Data Definition Directives

Directive	Full Form	Usage in Project
db	Define Byte (8-bit)	Storing strings, single bytes, buffer definitions
times N db 0	Repeat N times	Allocating zero-filled buffer space
[BITS 16]	Bits directive	Tells NASM to generate 16-bit machine code
[ORG 0x100]	Origin directive	Sets code start address (100h for COM files)

7.2 Data Movement Instructions

Instruction	Syntax	What It Does
MOV	mov dest, src	Copy value from src to dest (most used instruction)
XOR	xor reg, reg	XOR with itself = set register to 0 (faster than MOV reg,0)
PUSH	push reg/mem	Save value onto the stack (SP decreases by 2)
POP	pop reg/mem	Restore value from stack into register (SP increases by 2)

7.3 Arithmetic Instructions

Instruction	Syntax	What It Does
-------------	--------	--------------

ADD	add dest, src	dest = dest + src
SUB	sub dest, src	dest = dest - src
MUL	mul reg	AX = AL * reg (unsigned multiply)
DIV	div reg	AL = AX / reg, AH = AX % reg (unsigned divide)
INC	inc dest	dest = dest + 1
DEC	dec dest	dest = dest - 1
CMP	cmp a, b	Compute a - b, set flags (ZF, SF, CF), discard result

7.4 Jump Instructions

Instruction	Condition	What It Does
JMP label	Always	Unconditional jump to label
JE label	ZF = 1	Jump if equal (last CMP was equal)
JNE label	ZF = 0	Jump if not equal
JL label	SF != OF	Jump if less than (signed)
JG label	ZF=0 & SF=OF	Jump if greater than (signed)
JGE label	SF = OF	Jump if greater or equal
Instruction	Condition	What It Does
LOOP label	CX != 0	Decrement CX, jump if CX is not zero
CALL label	Always	Call subroutine: push return address, jump
RET	Always	Return from subroutine: pop return address, jump

7.5 String Instructions

Instruction	What It Does
-------------	--------------

MOVSb	Copy byte from [SI] to [DI], then SI++, DI++
REP MOVSb	Repeat MOVSb exactly CX times (bulk memory copy)

7.6 DOS Interrupt Functions (INT 21h)

AH Value	Function Name	Input	Output / Effect
01h	Read Char with Echo	None	AL = ASCII of key pressed, echoed to screen
02h	Write Char to Screen	DL = character	Prints the character in DL to screen
09h	Print String	DX = address of string ending with '\$'	Prints string to screen until '\$'
0Ah	Buffered Input	DX = buffer address	Reads full string from keyboard
4Ch	Exit Program	AL = exit code (0 = success)	Terminates program, returns to DOS

8. Output / Results

The following screenshots show the actual program output running in DOSBox.

8.1 Main Menu

```

+=====+
|  CIPHERASM - Caesar Cipher v2.0  |
|  NASM Assembly | COAL Project  |
+=====+

[1] Encrypt Text
[2] Decrypt Text
[3] Brute Force (All 25 Keys)
[4] Step-by-Step Encrypt
[5] Statistics of Text
[6] Exit

```

8.2 Encryption — PAKISTAN with Shift 2

```
[1] Encrypt Text
[2] Decrypt Text
[3] Brute Force (All 25 Keys)
[4] Step-by-Step Encrypt
[5] Statistics of Text
[6] Exit

Choice (1-6): 1

Enter text (max 50 chars): pakistan

Shift key (1-25): 2

[ORIGINAL ] : pakistan
[ENCRYPTED] : rcmkuvcp
```

8.3 Decryption — rcmkuvcp with Shift 2

This output was captured from the actual running program. The ciphertext 'rcmkuvcp' is successfully decrypted back to 'pakistan' using shift key 2:

```
+=====+
| CipherASM - Caesar Cipher v2.0 |
| NASM Assembly | COAL Project |
+=====+

[1] Encrypt Text
[2] Decrypt Text
[3] Brute Force (All 25 Keys)
[4] Step-by-Step Encrypt
[5] Statistics of Text
[6] Exit

Choice (1-6): 2

Enter text (max 50 chars): rcmkuvcp

Shift key (1-25): 2

[ORIGINAL ] : rcmkuvcp
[DECRYPTED] : pakistan
Run again? (Y/N): S
[!] Press Y or N only!
```

8.4 Step-by-Step Encryption — PAKISTAN with Shift 2

```
Enter text (max 50 chars): PAKISTAN
Shift key (1-25): 2

[STEP BY STEP ENCRYPTION]
-----
Char 'P' + shift = 'R'
Char 'A' + shift = 'C'
Char 'K' + shift = 'M'
Char 'I' + shift = 'K'
Char 'S' + shift = 'U'
Char 'T' + shift = 'V'
Char 'A' + shift = 'C'
Char 'N' + shift = 'P'
```

8.5 Text Statistics

```
[TEXT STATISTICS]
-----
Total Chars   : 8
Uppercase     : 0
Lowercase     : 8
Spaces        : 0
Others        : 0
```

9. Conclusion

SecureText successfully implements a fully functional Caesar Cipher Encryption and Decryption tool in 16-bit x86 Assembly Language. The project covers a comprehensive range of COAL concepts including register manipulation, memory addressing, interrupt-driven I/O, control flow with conditional jumps, procedure calls, stack usage, and string operations.

The program goes beyond a basic implementation by providing six distinct features: encryption, decryption, brute force cryptanalysis, step-by-step visualization, text statistics, and clean program exit. Each feature demonstrates practical application of assembly-level programming concepts that are core to the COAL curriculum.

The decryption feature was successfully verified: inputting the ciphertext 'rcmkuvcp' with shift key 2 correctly produces the original plaintext 'pakistan', confirming that the cipher and reverse-key logic work correctly at the machine level.

This project demonstrates that low-level assembly programming provides complete control over hardware resources and a deep understanding of how software interacts with the CPU. It reinforces the importance of COAL as a foundational subject in computer science and software engineering education.